

RMC_Knowledge_Base_v3

RMC — Reconfigurable DDR5 Memory Controller

Complete Project Knowledge Base v3

All locked decisions, architecture, tables, FSMs, scheduler pipeline

0. Project Identity

Item	Value
Project	Reconfigurable Memory Controller (RMC) — DDR5 capstone
Standard	JEDEC JESD79-5 (DDR5), DFI 5.2, AXI4
Target	4GB, single-channel, single-rank (anchor)
Future ceiling	46b address, 8ch, 8rank, 64Gb×3DS-8H = 64TB
RTL style	SystemVerilog, parameterized, no hardcoded widths

1. Locked Architecture Decisions

Decision	Value	Rationale
BL	BL16 only	AXI 64B-aligned INCR = one BL16 burst
AXI burst type	INCR only	WRAP/FIXED not useful for DRAM
AXI data width	64-bit	Sufficient for target
Narrow/unaligned	Not supported	Too complex, low benefit
Exclusive access	Not supported	Out of scope
QoS	Not supported	Not justified
Outstanding WR	64 (v1), 96 (v2)	2-3x RD entries
Outstanding RD	32	Lower latency priority
CDC crossing	Async FIFO only at AXI ↔ MC boundary	One verification target
Intra-MC interfaces	Valid-credit (no ready back-pressure)	Better timing closure

Decision	Value	Rationale
AXI interfaces	Valid-ready (spec mandated)	AXI4 compliance
Resp FIFO → CIF	Valid-only, MC self-throttles	No ready from CIF
Scheduler type	5-stage pipeline, TCAM-assisted, SJF	Industry standard
TCAM split	WR_TCAM (full addr) + RD_TCAM (BG/bank)	Separate search semantics
Page policy	Open default, closed/adaptive configurable	Open best for sequential
Maintenance Engine	Model A — peer block to Scheduler	Writes into FSM tables
Timing go/no-go	Registered can_* flags, timestamps stored	No subtractor in critical path
Bank pipelining	ACT → diff BG → ACT → CAS interleave	$t_{RRD_S} < t_{RRD_L}$
Starvation	Staggered: age $\geq$ THR + entry_idx	One miss/cycle, no collision
WR buffer larger	$N_{WR} = 2-3 \times N_{RD}$	Better RD latency, same bandwidth
RAW	WR_TCAM hit on new RD → RAW Bypass Manager	write-same-or-earlier = valid hit

2. Interface Protocol Summary

Interface	Protocol	Reason
AXI4 ports	Valid-ready	Spec mandated
Async req FIFO write (CIF)	Credit-based push	No combinational wr_full
Async req FIFO read (MC)	Valid-credit receive	registered rd_valid + credit_return
Async resp FIFO write (MC)	Credit-based push	gate_resp_fifo_avail guards sends
Async resp FIFO read (CIF)	Valid-credit receive	registered rd_valid + credit_return
Credit return req (MC → CIF)	Registered 1b sync	Crosses CDC safely
Credit return resp (CIF → MC)	Registered 1b sync	Crosses CDC safely
All intra-MC paths	Valid-credit	No combinational ready, better timing
DFI signals	DFI 5.2 protocol	Spec mandated

Interface	Protocol	Reason
PHY → MC rddata	Valid-only	No back-pressure on read return

Valid-Credit Interfaces (intra-MC)

```

watermark manager → TCAM alloc
TCAM hit          → Scheduler Stage 1
Stage 1           → Stage 2 → Stage 3 → Stage 4
Maintenance Eng   → Scheduler Stage 0
Hold-Forward      → Resp FIFO write port
Write Data Buf    → Write Data Path
Read Data Path    → Hold-Forward

```

3. System Block Diagram

```

AXI Masters (up to 32 clients)
  ↓ [AXI4 valid-ready]
AXI Interconnect
  ↓
CIF (AXI clock domain)
  ├── AXI Write Port / AXI Read Port    [valid-ready]
  ├── Burst Splitter
  ├── Address Translator
  ├── ROB
  └── Merge Logic
      ↓ [Async FIFO – CDC boundary, sole crossing]
MC Core (MC clock domain) ← all intra-MC = valid-credit
  ├── Config Registers (CSR via AXI4-Lite)
  ├── Init FSM (16 states)
  ├── Write Data Buffer (SRAM, 576b/entry)
  ├── WR_TCAM [N_WR_ENTRIES, full addr search]
  ├── RD_TCAM [N_RD_ENTRIES, BG/bank search]
  ├── WR Status Reg [valid, status, age]
  ├── RD Status Reg [valid, status, age]
  ├── Write Watermark Buffer Manager
  ├── Read Watermark Buffer Manager
  ├── RAW Bypass Manager (2-stage)
  ├── Hold-and-Forward (2-deep)
  ├── Per-Bank FSM Table (16×N_RANKS)
  ├── Per-Rank FSM Table (N_RANKS)
  ├── Global Timing Table
  ├── Bank Activity Counter Table (16×N_RANKS)
  ├── Global Cycle Counter (GC_WIDTH)
  └── timing_reg_file

```

```

└─ Scheduler (5 stages)
└─ Maintenance Engine (4 sub-FSMs)
└─ Write Data Path
└─ Read Data Path
└─ Error Handler
    ↓ [DFI 5.2]
DDR PHY → DDR5 DRAM

```

4. Buffer Sizing

Buffer	v1	v2	Rationale
N_WR_ENTRIES	64	96	2-3× RD, background WR drain
N_RD_ENTRIES	32	32	Lower latency priority
Write Data Buffer	32 entries	32 entries	576b/entry, SRAM
WR_TCAM	N_WR_ENTRIES	N_WR_ENTRIES	Sized to WR buffer
RD_TCAM	N_RD_ENTRIES	N_RD_ENTRIES	Sized to RD buffer

5. TCAM Split (Locked)

WR_TCAM

```

purpose: RAW detection (full address match)
search key: {BG, bank, row, col}    exact match
entry fields:
  bg          [BG_BITS]
  bank        [BANK_BITS]
  row         [ROW_BITS]
  col         [COL_BITS]
  req_type    1b = WR
  axi_id      [AXI_ID_WIDTH]
  entry_idx   [$clog2(N_WR_ENTRIES)] → WR status reg
  data_buf_idx [4:0]                  optional → Write Data Buffer

```

```

valid gating: WR_TCAM match[i] AND wr_status_reg[i].valid

```

RD_TCAM

```
purpose: scheduler bank lookup (BG/bank pre-filter)
search key: {BG, bank}    ternary, don't-care on row/col
entry fields:
  bg          [BG_BITS]
  bank        [BANK_BITS]
  row         [ROW_BITS]    carried, not matched
  col         [COL_BITS]    carried, not matched
  req_type    1b = RD
  axi_id      [AXI_ID_WIDTH]
  entry_idx   [$clog2(N_RD_ENTRIES)] → RD status reg
```

```
valid gating: RD_TCAM match[i] AND rd_status_reg[i].valid
multi-hit: winner = argmax(age[i]) via status reg
```

6. Status Register Fields (Locked)

WR Status Reg (N_WR_ENTRIES)

```
valid    1b
status   STATUS_WIDTH    00=PENDING 01=ISSUED 10=DONE 11=ERROR
age      [GC_WIDTH]      allocation timestamp
```

RD Status Reg (N_RD_ENTRIES)

```
valid    1b
status   STATUS_WIDTH    00=PENDING 01=ISSUED 10=DONE 11=ERROR
age      [GC_WIDTH]      allocation timestamp
```

owner: watermark managers (R/W). scheduler: READ ONLY.

valid gates TCAM match output — single source of truth for occupancy.

age used for: multi-hit newest-wins, starvation check, SJF cost.

7. RAW Bypass Manager

Hit valid condition

```
write arrived same cycle OR earlier than read
  wr_status_reg[hit_idx].age <= rd_age
if write arrived AFTER read → not a hit → normal mem_read
```

Stage A — WR_TCAM search

```

search key: {BG, bank, row, col}
hit vector gated by wr_status_reg[i].valid
multi-hit: newest age wins (argmax via status reg)

```

Stage B — mask coverage

```

coverable = (rd_mask & wr_mask) == rd_mask

full hit      → route wr_data → hold_forward
masked/overlap → route covered bytes → hold_forward
                overshoot bytes → DROP
partial gaps  → stall rd, wait wr retire
no hit / late → pass rd to scheduler

```

8. FSM State Tables (All Locked)

8A. Per-Bank FSM Table (16 × N_RANKS)

Field	Width	Description
state	BANK_STATE_WIDTH	Bank state
row_open	ROW_BITS	Open row address
next_cas	GC_WIDTH	CAS deadline timestamp
next_pre	GC_WIDTH	PRE deadline timestamp
next_act	GC_WIDTH	ACT deadline timestamp
next_ref	GC_WIDTH	REFsb deadline timestamp
can_cas	1b	Registered flag
can_pre	1b	Registered flag
can_act	1b	Registered flag
can_ref	1b	Registered flag
ref_pending	1b	Set by Maintenance Engine

```

can_* update (every cycle, background):
    can_act <= (gc - next_act)[GC_WIDTH-1] == 0
    can_pre <= (gc - next_pre)[GC_WIDTH-1] == 0
    can_cas <= (gc - next_cas)[GC_WIDTH-1] == 0
    can_ref <= (gc - next_ref)[GC_WIDTH-1] == 0
Stage 2 reads can_* only – no subtractor in critical path

```

States

Enc	State
000	IDLE
001	ACTIVATING
010	ACTIVE
011	PRECHARGING
100	REFRESHING_SB
101	RFM_ACTIVE
110	POWER_DOWN
111	SELF_REFRESH

Transitions

From	To	Trigger	Writeback
IDLE	ACTIVATING	ACT issued	next_cas=gc+tRCD, next_pre=gc+tRAS
ACTIVATING	ACTIVE	can_cas==1	row_open=req.row
ACTIVE	PRECHARGING	row miss, no hits	next_act=gc+tRP
ACTIVE	ACTIVE	CAS row hit	next_pre updated
PRECHARGING	IDLE	can_act==1	—
IDLE	REFRESHING_SB	ref_pending AND can_ref	next_ref=gc+tRFCsb
REFRESHING_SB	IDLE	can_ref==1	ref_pending=0
ACTIVE	RFM_ACTIVE	raa[b]<=RAAIMT	—
RFM_ACTIVE	IDLE	tRFM elapsed	raa reset
ANY	PD/SR	rank FSM	—
PD/SR	IDLE	rank FSM exit, can_act==1	—

8B. Per-Rank FSM Table (N_RANKS)

Field	Width
state	3b

Field	Width
next_rfc	GC_WIDTH
next_zq	GC_WIDTH
next_xp	GC_WIDTH
next_xs	GC_WIDTH
next_trefi	GC_WIDTH
next_zqcs	GC_WIDTH
can_rfc	1b
can_zq	1b
can_xp	1b
can_xs	1b
gate_rfc	1b
gate_zq	1b
ref_credits	4b
raa[16]	RAA_WIDTH × N_BANKS

States

Enc	State
000	NORMAL
001	REFab_ACTIVE
010	ZQCAL_ACTIVE
011	POWER_DOWN
100	SELF_REFRESH

8C. Global Timing Table (1 instance)

Field	Width
global_state	3b
next_act_any	GC_WIDTH
next_cas_any	GC_WIDTH
next_rd_wr	GC_WIDTH
next_wr_rd	GC_WIDTH

Field	Width
faw_window[4]	GC_WIDTH×4
next_act_bg[8]	GC_WIDTH×8
next_cas_bg[8]	GC_WIDTH×8
next_wtr_bg[8]	GC_WIDTH×8
last_act_bg[8]	GC_WIDTH×8
can_act_any	1b
can_cas_any	1b
can_rd_wr	1b
can_wr_rd	1b
can_faw	1b
can_act_bg[8]	N_BG bits
can_cas_bg[8]	N_BG bits
can_wtr_bg[8]	N_BG bits

8D. Bank Activity Counter Table (16 × N_RANKS)

Field	Width
count	$\$clog2(BUF_DEPTH+1)$
dirty	1b

9. Scheduler Pipeline (5 Stages)

Stage 0 — Maintenance Override

```
inputs:  ref_urgent, ref_due, zq_due, rfm_req, global_state
outputs: s0_override, me_cmd_{type,rank,bg,bank}
protocol: valid-credit from Maintenance Engine
```

Stage 1 — TCAM Search

```
inputs:  RD_TCAM hit bitmap + meta, rd_status valid+age
         WR_TCAM hit bitmap + meta, wr_status valid+age
```

outputs: s1_hit_bitmap[N_BANKS], s1_hit_meta[] per bank
protocol: valid-credit

Stage 2 — can_* Gate Check + Cost Classification

inputs: s1_hit_bitmap, s1_hit_meta
can_cas/pre/act/ref per bank
can_act/cas/faw global and per-BG
gate_rfc/gate_zq per rank
state, row_open per bank
outputs: HIT_SET bitmap, MISS_SET bitmap, remaining_cost[]
protocol: valid-credit

Stage 3 — SJF Winner Selection

inputs: HIT_SET, MISS_SET, remaining_cost[]
rd/wr status age[], gc
wr_count, wm hits, last_act_bg
outputs: winner_{valid,cmd_type,rank,bg,bank,row,col,entry_idx}
protocol: valid-credit

priority:

1. Stage 0 override
2. STARVED_MISS: age $\geq$ STARVATION_THR + entry_idx
3. HIT_SET: cost=0, prefer diff BG
4. MISS_SET: lowest remaining_cost

starvation: STARVATION_THR = $9 \times \text{tREFI}$ ~ 12480 cycles
stagger: at most one fires per cycle

Stage 4 — Emission + Writebacks

inputs: winner_*, s0_cmd_*, timing_reg_vals, gc
outputs: DFI signals
bank_fsm_update (state, next_*, row_open)
global_timing_update (next_*_any, bg arrays, faw)
status_update (entry_idx $\rightarrow$ ISSUED)
sched_ack $\rightarrow$ Maintenance Engine
raa_inc_en $\rightarrow$ Per-Rank FSM
protocol: valid-credit out to all update targets

10. Maintenance Engine (4 Sub-FSMs)

Priority (internal arbitration)

ref_urgent > ref_due > rfm_req > zq_due
one me_cmd_valid per cycle
protocol to scheduler: valid-credit

Refresh FSM (6 states)

IDLE→REF_DUE→WAIT_BANKS_IDLE→ISSUE_REF→WAIT_tRFC→DONE
leaky bucket: credits++ per tREFI, credits-- per REF
ref_urgent: credits >= 8
REFsb target: argmin(bank_act_count)
FGR-2x/4x: tRFC2/tRFC4, threshold halved/quartered
temp: MR4 TUF → tREFI/2 at >85C

ZQcal FSM (7 states)

IDLE→WAIT_IDLE→ISSUE_START→WAIT_tZQCAL
→ISSUE_LATCH→WAIT_tZQLAT→DONE
gate_zq[rank]=1 during entire sequence

RFM FSM (6 states)

IDLE→MONITOR_RAA→RFM_REQUEST→WAIT_ISSUE→WAIT_tRFM→UPDATE_RAA
raa[rank][bank]: +1 per ACT, -RAADec per REF
trigger: raa[b] <= RAAIMT

Power Management FSM (10 states)

PD: NORMAL→PD_ENTRY_CHECK→PRECHARGE_PD→ACTIVE_PD→PDX_WAIT→NORMAL
SR:
NORMAL→SR_ENTRY→WAIT_tCKSRE→SELF_REFRESHING→SR_EXIT→WAIT_tXS_tDLLK→NORMAL
PD entry: bank_act.count==0 AND no pending maintenance

11. Scheduler Invariants

1. row hit always beats row miss (unless starved)
2. max one ACT per cycle (faw_window enforces 4-ACT)
3. max one CAS per cycle
4. REF/ZQ/RFM override Stages 1-3
5. starved miss: at most one fires per cycle
6. RD never issues without gate_resp_fifo_avail
7. WR never issues without valid data_buf_idx
8. no cmd while gate_rfc[rank] or gate_zq[rank] asserted

9. consecutive ACTs prefer diff BG
10. CAS only when can_cas[b]==1
11. REFab: rank already idle, no bank-to-bank delay
12. REFSb: PRE required if target bank ACTIVE
13. SJF: lowest remaining_cost wins in MISS_SET
14. Stage 2 reads only can_* flags, no subtractor in critical path
15. WR_TCAM RAW hit valid only if wr_age <= rd_age
16. TCAM match suppressed when status_reg[i].valid==0

12. Legal Check Matrix (Stage 2)

Gate	Source	Note
can_cas[b]	Per-bank registered	tRCD elapsed
can_pre[b]	Per-bank registered	tRAS/tRTP/tWR elapsed
can_act[b]	Per-bank registered	tRP elapsed
can_ref[b]	Per-bank registered	tRFCsb elapsed
can_act_bg[bg]	Global registered	tRRD_L
can_act_any	Global registered	tRRD_S
can_faw	Global registered	≤4 ACTs in tFAW
can_cas_bg[bg]	Global registered	tCCD_L/tCCD_L_WR
can_cas_any	Global registered	tCCD_S
can_rd_wr	Global registered	tWTR_S
can_wr_rd	Global registered	tRTW
gate_rfc[r]	Per-rank	REFab blocking
gate_zq[r]	Per-rank	ZQcal blocking
gate_resp_fifo_avail	Resp FIFO	Free slot before RD

13. RAW Redirect (Locked)

Stage A: exact match {BG,bank,row,col} via WR_TCAM
gated by wr_status_reg[i].valid
multi-hit: argmax(age[i])

Stage B: mask coverage check
hit valid: wr_age <= rd_age
full hit → wr_data → hold_forward
overlap hit → covered bytes → hold_forward, overshoot DROP

partial gaps → stall rd until wr retires
no hit/late → scheduler

CRITICAL: RAW hit completes READ only

matched write untouched → full PENDING→ISSUED→DONE lifecycle

2-deep Hold-Forward: handles RAW hit + DRAM return same cycle

14. Pipeline Latency

Worked Example (@200MHz)

Path	Cycles
Row Hit	11
Row Empty	15
Row Miss	19
Full round-trip	~37
Timeout A (REF stall)	76

ROB watermark: 37. Option A locked (let CAS complete before REF).

15. Address Mapping

A[31:17] = Row A[16:14] = BG A[13:12] = Bank

A[11:2] = Col A[1:0] = Offset

Channel select = A[7] always (128B interleave)

BG/Bank bits fixed (3b+2b) across all configs

JEDEC ceiling: 41b. 46b with 3DS.

16. Config Registers (Key)

Register	Width	Default
WR_HIGH_WM	6b	16
WR_LOW_WM	6b	4
AGE_THR1	8b	64
AGE_THR2	8b	256

Register	Width	Default
STARVATION_THR	14b	12480
PAGE_POLICY	2b	00=Open
REF_MODE	2b	00=REFab
GC_WIDTH	—	GC_WIDTH
N_WR_ENTRIES	—	64/96
N_RD_ENTRIES	—	32

17. FSM Count Summary

FSM	States	Instances
Init FSM	16	1
Per-Bank FSM	8	16×N_RANKS
Per-Rank FSM	5	N_RANKS
Global FSM	5	1
Refresh FSM	6	1
ZQcal FSM	7	1
RFM FSM	6	1
Power Mgmt FSM	10	1
Write CRC FSM	4	1
ECS FSM	4	1

18. Ownership Summary

Block	Owner	Scheduler
WR Status Reg	Write WM Manager	READ ONLY
RD Status Reg	Read WM Manager	READ ONLY
WR_TCAM	Write WM Manager	READ ONLY
RD_TCAM	Read WM Manager	READ ONLY
Write Data Buffer	Write WM Manager	READ ONLY
Per-Bank FSM Table	Scheduler S4 + Maint	WRITE S4, READ S2
Per-Rank FSM Table	Maintenance Engine	READ S0

Block	Owner	Scheduler
Global Timing Table	Scheduler S4	READ S2
Bank Activity Counter	WM Managers	READ Maint+PwrMgmt
timing_reg_file	CSR (init)	READ ONLY
Global Cycle Counter	Free-running	READ all

19. Open Questions

Topic	Status
RAW Stage A latency	Combinational parallel with alloc → OPEN
3rd collision Hold-Forward	Rate-limited by construction → OPEN
Req FIFO credit depth	How many credits issued to CIF → OPEN
XOR address hashing	ADR-0002 deferred
Runtime DIMM discovery	Out of scope
TCAM area budget	Not estimated
tZQCS_interval default	CSR, TBD
PD_IDLE_THRESHOLD	CSR, TBD
MR4 TUF polling path	Needs MRR cmd path spec
REFsb bank coverage counter	5b per rank, OPEN